Create tables

sql

create table employee(emp_id **varchar**(n), emp_name **char**(n), street_no **varchar**(n), city **char**(n), **primary key**(emp_id));

\d employee

This creates the employee table with emp_id as the primary key, which means it is unique and not null. varchar holds variable-length text and char(n) holds fixed-length text. \d employee is a psql meta-command that describes the table's columns, types and indexes.

sql

create table company(company_name **varchar**(15), city **char**(15), **primary key** (company_name));

\d company

This creates company, with company_name as its primary key.

sql

create table works(emp_id **varchar**(n), company_name **varchar**(n), salary **numeric**(n,n), **primary key**(emp_id), **foreign key**(company_name) **references** company(company_name));

\d works

This links employees to companies. emp_id is the primary key, so each employee works at only one company. The foreign key means company_name must already exist in company. numeric(p,s) stores a number with p total digits and s decimal places.

sql

create table manages(emp_id **varchar**(15), manager_id **varchar**(15));

\d manages

This creates a table that maps each employee to their manager. It has no keys or constraints.

sql

alter table employee **alter column** emp_name **set** not null;

\d employee

This adds a NOT NULL constraint to emp_name. The second \d shows emp_name now marked "not null".

sql

alter table employee **add** age **numeric**(5);

This adds a new column age to employee. The table description shows it as numeric(5,0).

Screenshot 2: Alter tables

sql

\d employee

alter table employee **rename to** Employee1;

\d Employee1

This renames the table. PostgreSQL stores unquoted names in lowercase, so the table is actually named employee1.

sql

alter table company **alter column** city **type varchar**(20);

\d company

This changes the data type of city from char(15) to varchar(20).

sql

alter table company **drop** city;

\d company

This removes the city column and its data. The description now shows only company_name. It also lists the works foreign key that references it.

sql

alter table company **add** city **varchar**(20);

\d company

This adds city back, this time as varchar(20). It goes at the end of the column list.

Screenshot 3: Rename and drop

sql

create table Employee2(emp_id **varchar**(n), emp_name **char**(n), street_no **varchar**(n), city **char**(n), **primary key**(emp_name));

\d Employee2

This creates a second test table. Here the primary key is emp_name instead of emp_id, and the description shows emp_name as not null.

sql

alter table Employee1 **rename to** Employee;

\d Employee

This restores the original name. The description shows emp_id and emp_name as not null, plus the added age column.

sql

Drop table Employee2;

This deletes the Employee2 table and its data permanently.

sql

Drop table Employee1,company,works,manages;

This drops several tables in one statement. The table was renamed back to Employee in the previous step, so Employee1 no longer exists and this command will give an error. Use employee in place of Employee1 to drop it. company and works can be dropped together in one statement even though works depends on company.

creating and changing the table

Employee and company data

sql

```
insert into employee values('E-101','Adarsh','101','MG Road');

insert into employee values('E-102','Benny','101','MG Road');

insert into employee values('E-103','Catherine','102','Cochin');

insert into employee values('E-104','Glenn','201','Ernakulam');

insert into employee values('E-105','George','201','MG Road');

insert into employee values('E-106','Hayes','101','MG Road');

insert into employee values('E-107','Johnson','102','Cochin');

insert into employee values('E-108','James','101','Cochin');

insert into employee values('E-109','Karthik','101','Ernakulam');

insert into employee values('E-110','Lavanya','101','PalaceRoad');

insert into employee values('E-111','Niharika','102','Ernakulam');

select* from employee;
```

Each insert into ... values(...) adds one row. The values must follow the column order: emp_id, emp_name, street_no, city. Every insert prints INSERT 0 1, which means one row was added. select* from employee; displays all 11 rows.

Two of the inserts are hard to read in the screenshot. The Catherine insert looks like it says E-107, but the output shows E-103, and inserting E-107 twice would fail because emp_id is a primary key. I've also read the blurry Ma Road as MG Road.

sql

```
insert into company values('SBI','MG Road');

insert into company values('SBT','MG Road');

insert into company values('Federal','Broadway');

insert into company values('Indian Bank','Cochin');
```

insert into company **values**('SIB','Ernakulam');

insert into company **values**('HDFC','PalaceRoad');

insert into company **values**('Axis','Cochin');

insert into company **values**('City Bank','Ernakulam');

select* **from** company;

This adds 8 banks, each with a company_name and city. The select* shows them all. company_name is the primary key, so every name has to be unique.

Screenshot 2: Works and manages data

sql

insert into works **values**('E-101','SBI','71000');

insert into works **values**('E-102','SBI','90000');

insert into works **values**('E-103','SBT','40000');

insert into works **values**('E-104','Federal','37000');

insert into works **values**('E-105','SBT','17000');

insert into works **values**('E-106','Indian Bank','30000');

insert into works **values**('E-107','SIB','21000');

insert into works **values**('E-108','SIB','18000');

insert into works **values**('E-109','Indian Bank','28000');

insert into works **values**('E-110','SBT','25000');

insert into works **values**('E-111','Federal','40000');

select* **from** works;

These rows record which company each employee works for and their salary. Every company_name must already exist in company, because of the foreign key. If one doesn't, the insert is rejected. Salary is written as a quoted string, and PostgreSQL converts it to numeric.

sql

insert into manages **values**('E-101','E-102');

insert into manages **values**('E-102','Null');

insert into manages **values**('E-103','E-110');

insert into manages **values**('E-104','E-111');

insert into manages **values**('E-105','E-110');

insert into manages **values**('E-106','E-109');

insert into manages **values**('E-107','Null');

insert into manages **values**('E-108','Null');

insert into manages **values**('E-109','Null');

insert into manages **values**('E-110','Null');

insert into manages **values**('E-111','Null');

Each row gives an employee and their manager. For example, E-101 is managed by E-102. The value 'Null' in quotes is the text string "Null", not a real SQL NULL. That works here because manager_id is a varchar with no constraint. If you want a true NULL, write null without quotes.

Screenshot 3: Manages output

sql

select* **from** manages;

This shows all 11 rows of manages. E-101 reports to E-102. E-103 and E-105 report to E-110. E-104 reports to E-111. E-106 reports to E-109. The other employees show Null, meaning they have no manager listed.

These pictures contain **SQL queries**, mainly using SELECT, WHERE, JOIN, GROUP BY, HAVING, aggregate functions, and UPDATE.

1. Find an employee by city

sql

```
select emp_name from employee where city = 'Palace Road';
```

Displays the names of employees whose city is Palace Road.

- select emp_name chooses only the employee name.
- from employee reads from the employee table.
- where city = 'Palace Road' filters the rows.

Output:

text

Lavanya

2. Employees who work for SBT

sql

```
select emp_name from emp, works
where emp.emp_id = works.emp_id
and companyname = 'SBI';
```

This query combines employee and works to display employees working for SBI.

The picture appears to show emp, but your table was created as employee. The clearer/correct version is:

sql

```
select employee.emp_name
from employee, works
where employee.emp_id = works.emp_id
and works.company_name = 'SBI';
```

employee.emp_id = works.emp_id connects the two tables.

Output:

text

Adarsh

Benny

George

Lavanya

3. Employees working for SBI or SBT

sql

```
select emp_name
from employee, works, company
where employee.emp_id = works.emp_id
and works.company_name = company.company_name
and city = 'SBI'
and company.city = 'Cochin';
```

The picture is slightly blurry. The intended query appears to connect all three tables and filter by a company/city condition. A safer version for employees working for SBI is:

sql

```
select employee.emp_name
from employee, works, company
where employee.emp_id = works.emp_id
and works.company_name = company.company_name
and works.company_name = 'SBI';
```

This returns employees whose company is SBI.

4. Employee and manager names

sql

```
select emp.emp_name, m.emp_name as managername
from emp, manages m
```

where emp.emp_id = m.emp_id

and m.manager_id = m.emp_id;

The image is blurry here. The intended query is a self-join between the employee table and the manages table. The correct version is:

sql

select e.emp_name, manager.emp_name **as** managername

from employee e, manages m, employee manager

where e.emp_id = m.emp_id

and m.manager_id = manager.emp_id;

This displays each employee together with their manager's name.

- e represents the employee.
- manager represents the same employee table a second time.
- as managername gives the second output column a readable name.

5. Employees earning more than 70,000

sql

select emp_name, emp_id, salary

from works, employee

where salary > 70000

and works.emp_id = employee.emp_id;

This displays the employee name, ID and salary for employees earning more than 70,000.

A clearer version is:

sql

select employee.emp_name, employee.emp_id, works.salary

from employee, works

where works.salary > 70000

and works.emp_id = employee.emp_id;

6. Salary paid by each company

sql

select companyname, sum(salary) **as** salary_paid

from works

group by companyname;

The picture appears to use companyname; according to your table, the correct column is company_name:

sql

select company_name, sum(salary) **as** salary_paid

from works

group by company_name;

SUM(salary) adds all salaries for each company. GROUP BY company_name creates one result row per company.

7. Companies with total salary above 50,000

sql

select company_name, sum(salary)

from works

group by company_name

having sum(salary) > 50000;

This calculates the total salary paid by each company and displays only companies whose total is greater than 50,000.

WHERE filters individual rows, while HAVING filters groups created by GROUP BY.

8. Salary paid by SBI

sql

select sum(salary)

from works

where company_name = 'Indian Bank'

group by company_name;

The picture appears to show Indian Bank, although the query heading/output may refer to SBI. To calculate SBI's total salary, use:

sql

select sum(salary)

from works

where company_name = 'SBI'

group by company_name;

To calculate Indian Bank's total salary, use:

sql

select sum(salary)

from works

where company_name = 'Indian Bank'

group by company_name;

9. Increase salaries

sql

update works

set salary = salary * 1.1

where emp_id in (

 select manager_id

 from manages

);

This increases the salary of employees who are managers by 10%.

- salary * 1.1 adds 10%.
- The subquery finds employee IDs that appear as managers.
- update works changes the salary values in works.

The picture may show salary = salary + 1, but the output indicates a 10% increase, so salary = salary * 1.1 is the meaningful command.

10. Display updated salaries

sql

```
select * from works;
```

Displays every column and row from the works table after the salary update.

11. Companies with more than one employee

sql

```
select companyname
from works
group by companyname
having count(distinct emp_id) > 1;
```

Using your column name, the corrected command is:

sql

```
select company_name
from works
group by company_name
having count(distinct emp_id) > 1;
```

This lists companies that employ more than one different employee.

- COUNT(DISTINCT emp_id) counts unique employees.
- HAVING ... > 1 keeps only companies with at least two employees.

12. Companies with an employee named Adarsh

sql

```
select company_name
from works
where company_name in (
```

```
 select company_name
 from works, employee
 where works.emp_id = employee.emp_id
 and emp_name = 'Adarsh'
)
group by company_name;
```

A simpler equivalent query is:

```
sql
select works.company_name
from works, employee
where works.emp_id = employee.emp_id
and employee.emp_name = 'Adarsh';
```

This finds the company where Adarsh works.

13. Employees earning more than the average salary

```
sql
select emp_name, salary
from works, employee
where salary > (
 select avg(salary)
 from works
)
and works.emp_id = employee.emp_id;
```

This displays employees whose salary is higher than the average salary of all employees.

- The inner query calculates the average salary.
- The outer query compares each employee's salary with that average.
- The join obtains the employee name from employee.

The picture appears to use Indian Bank in the subquery in one place. If the intended question is “employees earning more than the average salary at Indian Bank,” use:

sql

```
select employee.emp_name, works.salary
from works, employee
where works.salary > (
  select avg(salary)
  from works
  where company_name = 'Indian Bank'
)
and works.emp_id = employee.emp_id;
```

These four pages contain SQL queries for a **student–subject–faculty database**. The commands below follow the order in the pictures. Some names are slightly blurry, so I have used the most likely spelling from the visible output.

1. Display faculty names and IDs

sql

select f_name, f_code

from faculty;

Displays all faculty members and their faculty codes.

- f_name is the faculty name.
- f_code is the faculty ID.
- from faculty specifies the table.

2. Display all subjects

sql

select * **from** subject;

Displays every column and row in the subject table.

The output contains columns such as:

text

subjectcode | subjectname | maxmark | faculty_code

3. Display all students

sql

select * **from** student;

Displays all records from the student table, including student code, name, date of birth, branch and admission date.

4. Display student marks

sql

select * **from** studentmark;

Displays all records from the studentmark table.

The output contains:

text

studentcode | subjectcode | mark

This table stores the mark obtained by every student in each subject.

5. Count faculty members

sql

select count(f_code) **as** facultycount

from faculty;

Counts the number of faculty records.

- count(f_code) counts the faculty codes.
- as facultycount gives the result a heading.

6. Calculate total marks

sql

select studentname, sum(mark) **as** totalmark

from student, studentmark

where student.studentcode = studentmark.studentcode

group by studentname;

Calculates the total marks obtained by each student.

The condition:

sql

student.studentcode = studentmark.studentcode

connects the student table with the studentmark table.

sum(mark) adds all subject marks for each student.

7. Calculate average marks for each subject

sql

select subjectname, avg(mark) **as** avgmark

from subject, studentmark

where subject.subjectcode = studentmark.subjectcode

group by subjectname;

Calculates the average mark for each subject.

The tables are connected using:

sql

subject.subjectcode = studentmark.subjectcode

avg(mark) calculates the average mark.

8. Count students who took each subject

sql

select subject.subjectname,

count(student.studentname)

from subject, student, studentmark

where subject.subjectcode = studentmark.subjectcode

and student.studentcode = studentmark.studentcode

group by subject.subjectname;

Counts how many students have marks recorded for every subject.

The query joins three tables:

- subject provides the subject name.
- studentmark connects students and subjects.
- student provides the student name.

9. Find student percentages

sql

select studentname, subjectname, mark, maxmark,

round((studentmark.mark / subject.maxmark) * 100, 2) **as** percentage

from student, subject, studentmark

where student.studentcode = studentmark.studentcode

and subject.subjectcode = studentmark.subjectcode;

Calculates each student's percentage for every subject.

The formula is:

$$\text{percentage} = \frac{\text{mark}}{\text{maxmark}} \times 100$$

round(..., 2) rounds the percentage to two decimal places.

10. Find faculty members teaching a subject

sql

select f_name, subjectname

from faculty, subject

where faculty.f_code = subject.faculty_code;

Displays the faculty member and the subject assigned to that faculty member.

The tables are connected through:

sql

faculty.f_code = subject.faculty_code

11. Find faculty members teaching more than one subject

sql

select f_name

from faculty

where (

 select count(subjectcode)

 from subject

 where subject.faculty_code = faculty.f_code

) > 1;

Finds faculty members who teach more than one subject.

The inner query counts the subjects assigned to each faculty member. The outer query keeps only those whose count is greater than 1.

An equivalent GROUP BY version is:

sql

select faculty.f_name

from faculty, subject

where faculty.f_code = subject.faculty_code

group by faculty.f_name

having count(subject.subjectcode) > 1;

12. Display students sorted by marks

sql

select studentname, subjectname, mark

from student, subject, studentmark

where student.studentcode = studentmark.studentcode

and subject.subjectcode = studentmark.subjectcode

order by mark;

Displays student name, subject name and mark.

sql

order by mark

sorts the result from the lowest mark to the highest mark.

To sort from highest to lowest, use:

sql

order by mark **desc**;

Important terms

SQL command	Purpose
SELECT	Chooses the columns to display.
FROM	Specifies the table or tables.
WHERE	Filters rows or connects tables.
GROUP BY	Creates groups for calculations.
COUNT()	Counts rows or values.
SUM()	Adds numeric values.
AVG()	Calculates an average.
ROUND()	Rounds a number to a specified number of decimal places.
ORDER BY	Sorts the output.
AS	Gives a column or calculation a temporary name.

These pages contain queries using **JOIN**, **INTERSECT**, **subqueries**, and aggregate functions. I have listed them in the order shown.

1. Display complete student subject details

sql

select s.studentname, s.studentbranch, s.f_name, s.mark

from student s

join subject sub **on** s.subjectcode = sub.subjectcode

join studentmark m **on** s.studentcode = m.studentcode

join faculty f **on** sub.faculty_code = f.f_code;

This combines the student, subject, student-mark and faculty information.

- s, sub, m and f are aliases.
- studentcode connects students with their marks.
- subjectcode connects marks with subjects.
- faculty_code connects subjects with faculty members.
- The result displays student name, branch, faculty name and mark.

The picture may show slightly different aliases, but the purpose is the same.

2. Find students whose subject is taught by faculty

sql

select distinct s.studentname, s.studentbranch

from student s

join studentmark m **on** s.studentcode = m.studentcode

join subject sub **on** m.subjectcode = sub.subjectcode

join faculty f **on** sub.faculty_code = f.f_code;

This displays each student's name and branch.

distinct prevents duplicate students from appearing when a student has marks for multiple subjects.

3. Find the average mark for every subject

sql

```
select sub.subjectname, f.f_name, avg(m.mark) as avgmark
from studentmark m
join subject sub on m.subjectcode = sub.subjectcode
join faculty f on sub.faculty_code = f.f_code
group by sub.subjectname, f.f_name;
```

This calculates the average mark for each subject and displays the faculty member responsible for it.

- avg(m.mark) calculates the average.
- group by creates one group for each subject and faculty member.
- as avgmark gives the calculated column a heading.

4. Find students scoring above the subject average

sql

```
select studentname, subjectname, mark
from student s
join studentmark m on s.studentcode = m.studentcode
join subject sub on m.subjectcode = sub.subjectcode
where mark > (
  select max(mark)
  from studentmark
  where subjectcode = sub.subjectcode
);
```

The picture appears to use a subquery comparing marks with a subject-related value. However, to find students scoring above the **average** for their subject, the correct query is:

sql

```
select s.studentname, sub.subjectname, m.mark
from student s
```

join studentmark m **on** s.studentcode = m.studentcode

join subject sub **on** m.subjectcode = sub.subjectcode

where m.mark > (

select avg(m2.mark)

from studentmark m2

where m2.subjectcode = m.subjectcode

);

The inner query calculates the average mark for the same subject. The outer query displays only marks higher than that average.

5. Display faculty names in alphabetical order

sql

select f_name **as** name

from faculty

union

select studentname

from student;

The picture appears to show:

sql

select f_name **as** name **from** faculty **union select** studentname **from** student;

This combines faculty names and student names into one result.

- UNION combines two SELECT results.
- Both queries must return the same number of columns.
- UNION removes duplicate names.

If you want to display only faculty names alphabetically, use:

sql

select f_name

from faculty

order by f_name;

6. Find faculty codes present in both tables

sql

select f_code

from faculty

intersect

select faculty_code

from subject;

This finds faculty codes that occur in both the faculty table and the subject table.

- The first query gets all faculty codes.
- The second gets faculty codes assigned to subjects.
- INTERSECT returns only common values.

7. Find subjects that have marks

sql

select distinct subjectname

from subject

except

select subjectname

from subject

where subjectcode not in (

select subjectcode

from studentmark

);

The picture shows an EXCEPT query involving subjects and subject codes. The simpler and clearer version is:

sql

```
select distinct sub.subjectname
from subject sub
join studentmark m
on sub.subjectcode = m.subjectcode;
```

This displays subjects for which at least one mark has been recorded.

If you want to use EXCEPT to find subjects with no marks:

sql

```
select subjectname
from subject
except
select sub.subjectname
from subject sub
join studentmark m
on sub.subjectcode = m.subjectcode;
```

Important concepts

Command	Meaning
Command	**Meaning**
JOIN	Combines related rows from different tables.
ON	Specifies how the tables are connected.
DISTINCT	Removes duplicate results.
AVG()	Calculates an average.

Command	Meaning
GROUP BY	Groups rows before an aggregate calculation.
UNION	Combines results and removes duplicates.
INTERSECT	Returns values found in both queries.
EXCEPT	Returns values from the first query that are absent from the second.
Subquery	A query placed inside another query.
Alias	A short name such as s, m or sub for a table.

This picture demonstrates **views** in PostgreSQL. A view is a saved SELECT query that behaves like a virtual table.

1. Create the After2020 view

sql

```
CREATE VIEW StudentsAfter2020 AS
SELECT studentcode,
    studentname,
    studentbranch,
    date
FROM student
WHERE date > '2020-01-31';
```

This creates a view named StudentsAfter2020.

It selects students whose admission date is after 2020-01-31.

- CREATE VIEW creates a virtual table.
- AS introduces the query used by the view.
- WHERE date > '2020-01-31' filters the students.
- The original student table is not changed.

To display the view:

sql

```
SELECT * FROM StudentsAfter2020;
```

The output contains the students admitted after the specified date.

The date column in the picture may be named date, admissiondate, or something similar. Use the exact column name from your student table.

2. Create the HighMarkSubjects view

sql

```
CREATE VIEW HighMarkSubjects AS
SELECT subjectcode,
```

subjectname,

maxmark,

faculty_code

FROM subject

WHERE subject.maxmark > 100;

This creates a view containing subjects whose maximum mark is greater than 100.

To display it:

sql

SELECT * **FROM** HighMarkSubjects;

The view shows:

text

subjectcode | subjectname | maxmark | faculty_code

The condition:

sql

WHERE subject.maxmark > 100

keeps only subjects with a maximum mark above 100.

3. Create the SubjectHighestMarks view

sql

CREATE VIEW SubjectHighestMarks **AS**

SELECT subjectmark.subjectcode,

subjectname,

MAX(mark) **AS** HighestMark

FROM studentmark

JOIN subject

ON subjectmark.subjectcode = subject.subjectcode

GROUP BY subjectmark.subjectcode, subjectname;

This creates a view containing the highest mark obtained in every subject.

To display it:

sql

SELECT * **FROM** SubjectHighestMarks;

Explanation

sql

MAX(mark)

finds the largest mark.

sql

JOIN subject

ON subjectmark.subjectcode = subject.subjectcode

connects the marks table with the subject table so that the subject name can be displayed.

sql

GROUP BY subjectmark.subjectcode, subjectname

creates one group for each subject. Without GROUP BY, PostgreSQL could calculate only one maximum value for the entire table rather than one value per subject.

The result contains:

text

subjectcode | subjectname | highestmark

Complete commands together

sql

CREATE VIEW StudentsAfter2020 **AS**

SELECT studentcode,

    studentname,

    studentbranch,

    date

```
FROM student
WHERE date > '2020-01-31';

SELECT * FROM StudentsAfter2020;


CREATE VIEW HighMarkSubjects AS
SELECT subjectcode,
    subjectname,
    maxmark,
    faculty_code
FROM subject
WHERE subject.maxmark > 100;

SELECT * FROM HighMarkSubjects;


CREATE VIEW SubjectHighestMarks AS
SELECT studentmark.subjectcode,
    subjectname,
    MAX(mark) AS HighestMark
FROM studentmark
JOIN subject
ON studentmark.subjectcode = subject.subjectcode
GROUP BY studentmark.subjectcode, subjectname;
```

SELECT * **FROM** SubjectHighestMarks;

View versus table

Object	**Meaning**
Table	Stores actual data.
View	Stores a query, not a separate copy of the data.
CREATE VIEW	Saves a query as a virtual table.
SELECT * FROM view_name	Displays the view result.

If the view already exists, PostgreSQL will show an error. To remove it, use:

sql

DROP VIEW StudentsAfter2020;

DROP VIEW HighMarkSubjects;

DROP VIEW SubjectHighestMarks;